

WEB SECURITY & VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT)

Project Report

Week 6 — VAPT Basics

Prepared by	Khushveer
Company	DG Interns Hub
Course / Program	Cybersecurity Internship
Target Application	DVWA (Damn Vulnerable Web Application)
Submission Date	14th September 2026

Table of Contents

1. Introduction to VAPT	1
1.1 What is VAPT?	1
1.2 Why VAPT Matters	1
1.3 VA vs PT — Key Differences	1
1.4 Scope of This Project	1
2. Tools Used	1
2.1 Environment Setup	2
2.2 Proxy Configuration	2
3. Testing Process	2
3.1 SQL Injection Testing	2
3.2 Cross-Site Scripting (XSS) Testing	4
3.3 Broken Authentication Testing	7
3.4 Security Misconfiguration Review	8
4. Automated Scanning with OWASP ZAP	8
5. Findings & Risk Levels	9
5.1 Severity Rating Criteria	10
6. Mitigation Methods	10
6.1 SQL Injection	10
6.2 Cross-Site Scripting (XSS)	10
6.3 Broken Authentication	10
6.4 Security Misconfiguration	11
7. Conclusion	11
8. References	11

1. Introduction to VAPT

1.1 What is VAPT?

Vulnerability Assessment and Penetration Testing (VAPT) is a combined security testing approach used to identify, quantify, and address security weaknesses in an application, network, or system. Vulnerability Assessment (VA) focuses on discovering and cataloguing as many vulnerabilities as possible, typically using automated scanners. Penetration Testing (PT) goes a step further by actively exploiting selected vulnerabilities to demonstrate real-world impact and validate that the risk is genuine rather than a false positive.

1.2 Why VAPT Matters

- Identifies security weaknesses before malicious attackers can exploit them.
- Helps organizations comply with security standards and regulations (e.g., PCI-DSS, ISO 27001).
- Provides a prioritized, risk-based view of vulnerabilities so remediation effort is spent where it matters most.
- Validates that existing security controls (firewalls, input validation, authentication) actually work under attack conditions.

1.3 VA vs PT — Key Differences

Vulnerability Assessment is broad, automated, and non-intrusive — it answers 'what could be wrong?'. Penetration Testing is narrow, manual/semi-manual, and intrusive — it answers 'can this actually be exploited, and what happens if it is?'. A complete VAPT engagement uses both: automated scanning (OWASP ZAP) to cover breadth, and manual testing (Burp Suite + manual payloads) to confirm depth and real impact.

1.4 Scope of This Project

This project performs a VAPT exercise against DVWA (Damn Vulnerable Web Application), a deliberately vulnerable PHP/MySQL web application designed for learning web application security. The testing focuses on four vulnerability classes from the OWASP Top 10: SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Security Misconfiguration.

2. Tools Used

Tool	Purpose	Type
Burp Suite Community Edition	Intercepting proxy - capture, inspect, and replay HTTP requests/responses	Manual testing
OWASP ZAP	Automated vulnerability scanner - spidering and active scanning	Automated scanning
DVWA	Intentionally vulnerable web application used as the test	Target application

Tool	Purpose	Type
	target	
XAMPP / Docker	Local web server + MySQL stack to host DVWA	Environment

2.1 Environment Setup

DVWA was deployed locally on a Kali Linux testing environment. The PHP/MariaDB stack was configured, the DVWA database was initialized via the setup page, and the application was accessed at the local address 127.8.0.1 for all subsequent testing.

2.2 Proxy Configuration

Burp Suite and OWASP ZAP were both configured as intercepting proxies on the browser, with the respective CA certificates installed to allow interception of traffic to the local DVWA instance (accessed at 127.8.0.1 in this environment).

3. Testing Process

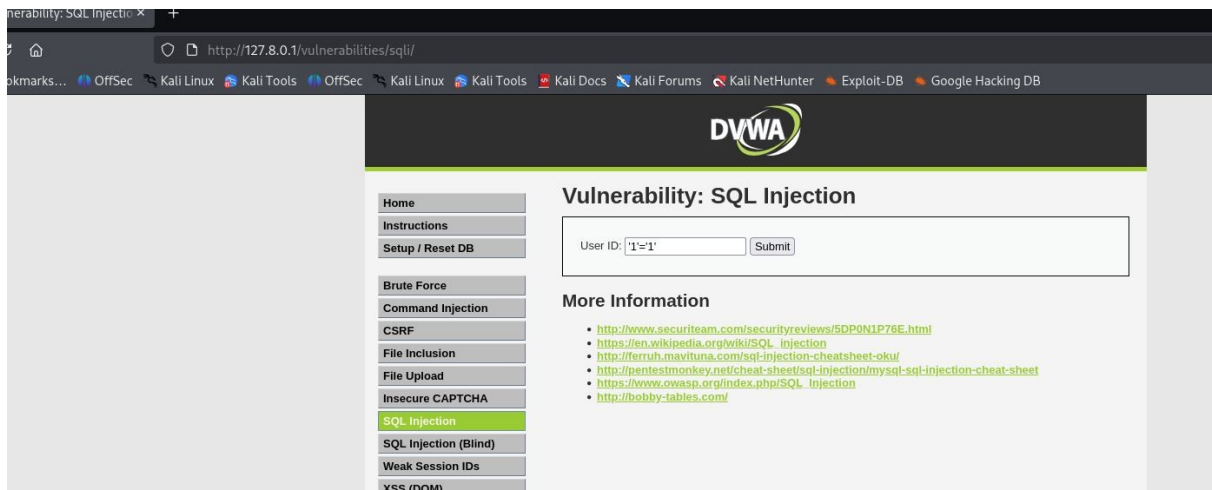
3.1 SQL Injection Testing

Testing was performed on the DVWA 'SQL Injection' module. The User ID input field, which is passed directly into a backend SQL query, was tested with a crafted payload to check whether user input was being sanitized before reaching the database.

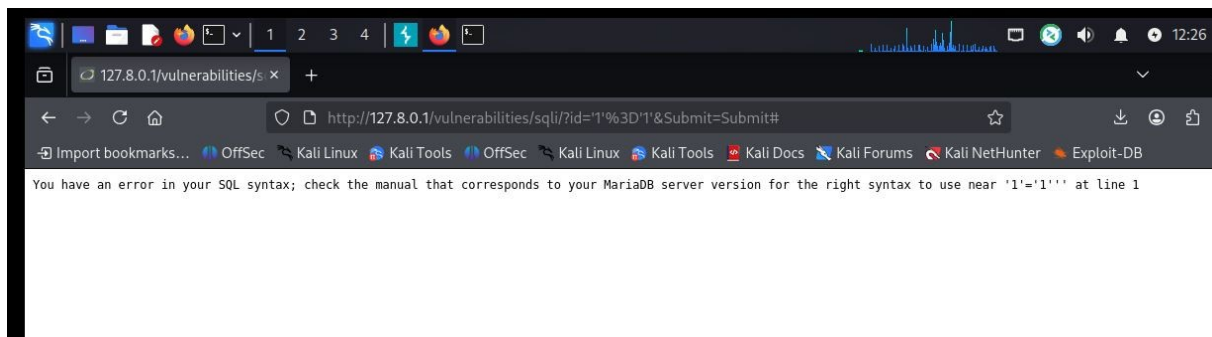
3.1.1 Error-based Injection

Payload used: '1'='1'

Submitting this unescaped payload directly into the User ID field broke the structure of the underlying SQL query. Instead of returning a result, the application returned a raw database error, confirming that user input is passed to the query without sanitization or parameterization.



Payload '1'=1' entered into the DVWA SQL Injection User ID field



Resulting SQL syntax error, confirming the injected input reached the backend query unsanitized

The error message itself is a finding: it discloses that the backend database is MariaDB and echoes back the exact fragment of the broken query, which an attacker can use to iteratively craft a working injection (e.g. a UNION-based payload to extract data from the users table).

3.1.2 Capturing the Request in Burp Suite

With the browser proxy routed through Burp Suite, the SQL Injection request was intercepted and reviewed in the Proxy > HTTP history tab. The raw request (GET /vulnerabilities/sqli/?id='1'=1'&Submit=Submit) and the corresponding response containing the SQL error were captured as evidence.

Filter settings: Hiding CSS and image content; hiding specific extensions Pro version or

Host	Method	URL ^	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
http://127.8.0.1	GET	/login.php			200	1851	HTML	php	Login :: Damn Vulnera...	
http://127.8.0.1	POST	/login.php		✓	302	337	HTML	php		
http://127.8.0.1	GET	/login.php			200	1889	HTML	php	Login :: Damn Vulnera...	
http://127.8.0.1	POST	/login.php		✓	302	337	HTML	php		
https://ads.mozilla.org	POST	/v1/ads		✓	200	8115	JSON			
https://ads.mozilla.org	POST	/v1/ads		✓	200	8198	JSON			
https://ads-img.mozilla.org	GET	/v1/images?image_data=CnAKbm...		✓	304	283				
https://ads-img.mozilla.org	GET	/v1/images?image_data=CnAKbm...		✓	304	283				
http://127.8.0.1	GET	/vulnerabilities/sqli/			200	4807	HTML		Vulnerability: SQL Inje...	
http://127.8.0.1	GET	/vulnerabilities/sqli/?id=%271%27%30%271%27&Submit=Submit		✓	200	504	HTML			
http://127.8.0.1	GET	/vulnerabilities/xss_s/			200	5320	HTML		Vulnerability: Stored C...	
http://127.8.0.1	POST	/vulnerabilities/xss_s/		✓	200	5416	HTML		Vulnerability: Stored C...	

equest

etty
Raw
Hex

```

GET /vulnerabilities/sqli/?id=%271%27%30%271%27&Submit=Submit
HTTP/1.1
Host: 127.8.0.1
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document

```

Response

Pretty
Raw
Hex
Render

```

1 HTTP/1.1 200 OK
2 Date: Sun, 13 Sep 2026 09:29:08 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 166
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <pre>
    You have an error in your SQL syntax; check the manual that
    corresponds to your MariaDB server version for the right
    syntax to use near '1='1'' at line 1
  </pre>

```

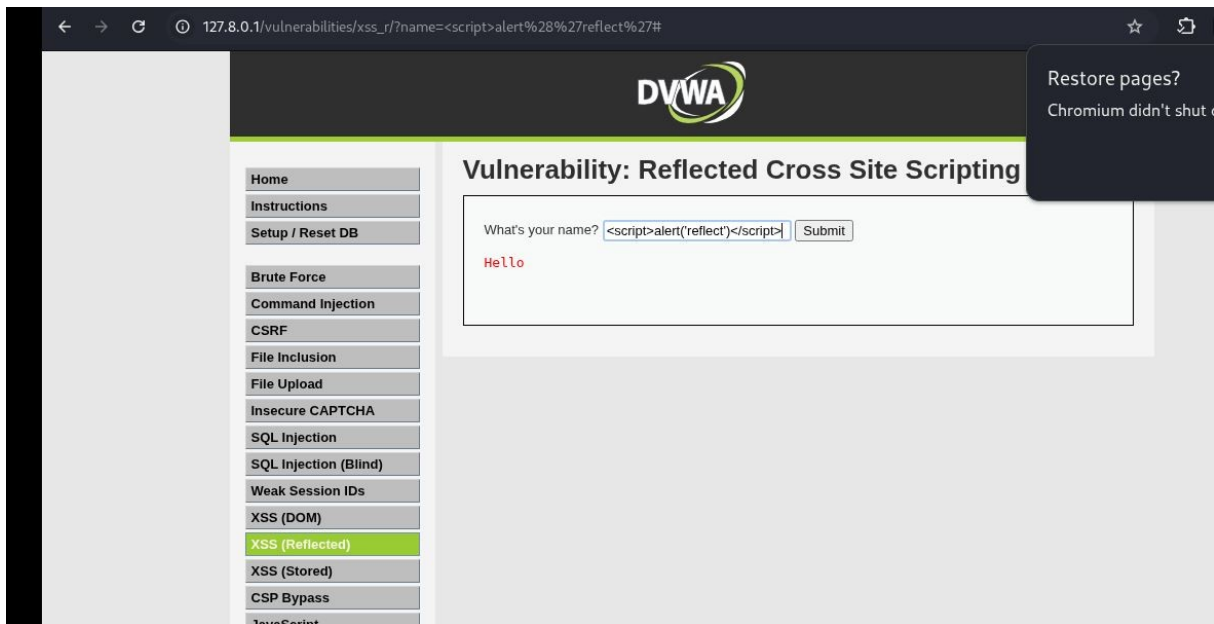
Burp Suite Proxy — HTTP history showing the SQL Injection request and the raw error response

3.2 Cross-Site Scripting (XSS) Testing

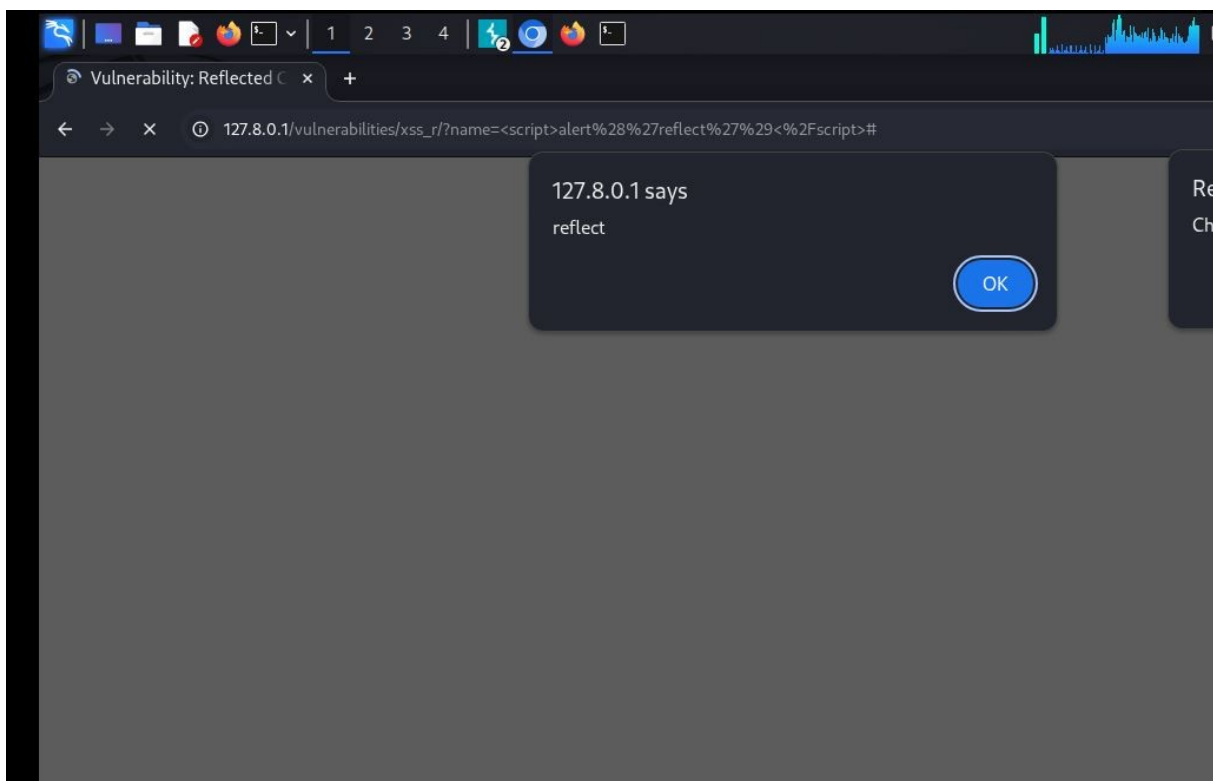
3.2.1 Reflected XSS

Tested on the DVWA 'XSS (Reflected)' module. Payload used: `<script>alert('reflect')</script>`

The payload was submitted via the name parameter and was immediately reflected back into the page response, executing the JavaScript alert in the browser. The payload is not stored server-side — it only affects the current request/response cycle, meaning a victim would need to click a specially crafted link containing the payload.



Reflected XSS payload entered into the 'What's your name?' field on DVWA

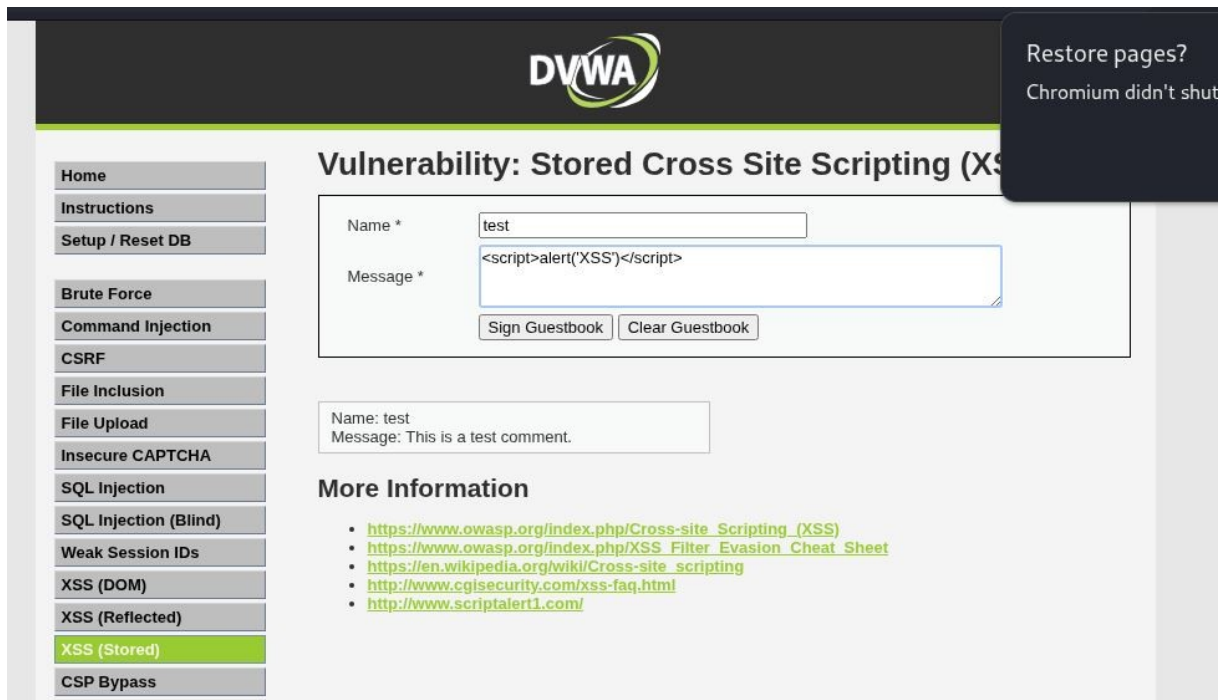


JavaScript alert box firing with the injected 'reflect' message, confirming script execution

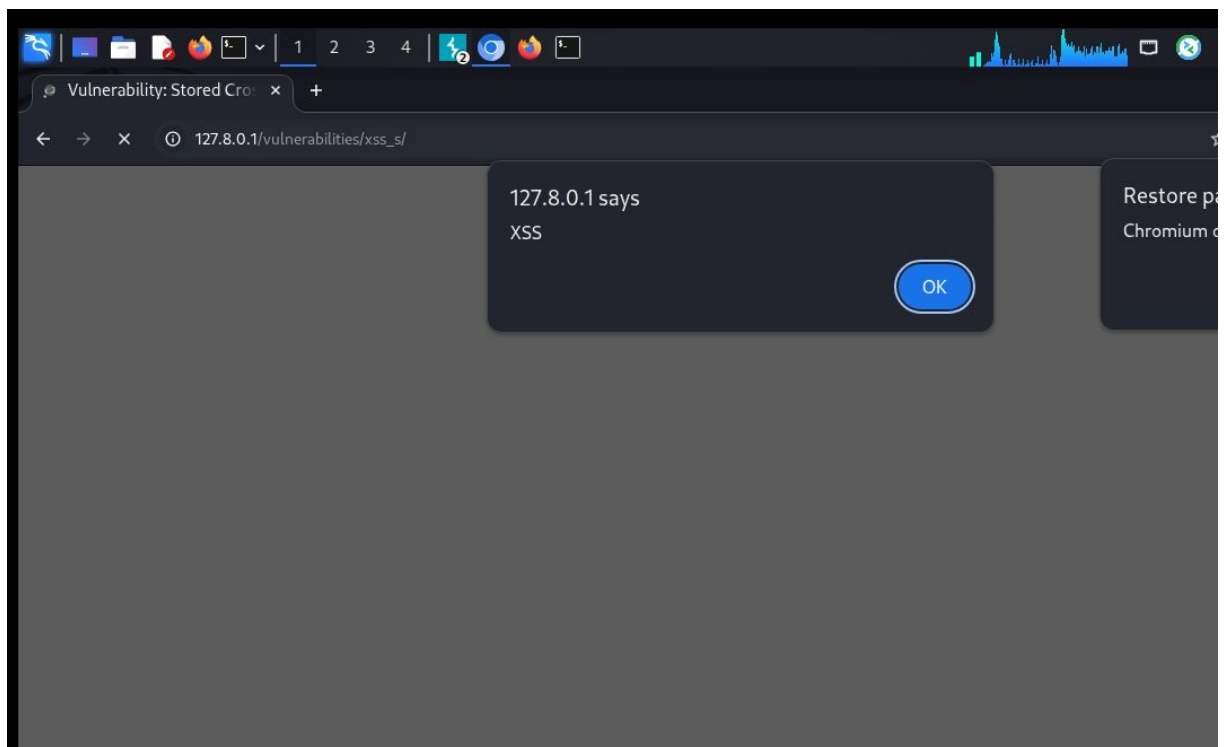
3.2.2 Stored XSS

Tested on the DVWA 'XSS (Stored)' module (guestbook). Payload used: `<script>alert('XSS')</script>`, submitted into the Message field.

Unlike the reflected case, this payload was persisted in the application's database. The alert fired automatically on page load/reload — demonstrating the higher severity of stored XSS, since it affects every visitor who views the guestbook rather than only the user who submitted the crafted request.



Stored XSS payload submitted into the DVWA guestbook Message field



JavaScript alert box firing automatically on page load, confirming the payload was persisted and executed

3.2.3 Capturing XSS Requests in Burp Suite

Both XSS submissions were captured in Burp's HTTP history to document the exact parameter names and request structure used to deliver each payload to the server.

The screenshot shows the Burp Suite interface with the Proxy tab selected. The HTTP history table lists several requests, with the selected request being a GET to `/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27reflect%27%2A%3C%2Fscript%3E HTTP/1.1`. The detailed view of this request and response is shown below the table.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time
19	https://ads-img.mozilla.org	GET	/v1/images?image_data=CnAKbm...		✓	304	283					✓	151.101.1.91		15:0
20	https://ads-img.mozilla.org	GET	/v1/images?image_data=CnAKbm...		✓	304	283					✓	151.101.1.91		15:0
27	https://safebrowsing.google.com	GET	/v4/threatListUpdates:fetch?Sct=...		✓	200	5178	app				✓	142.250.193.138		15:1
15	http://127.8.0.1	GET	/vulnerabilities/sqli/			200	4807	HTML		Vulnerability: SQL Inje...			127.8.0.1		14:5
16	http://127.8.0.1	GET	/vulnerabilities/sqli/?id=%27%27...		✓	200	504	HTML					127.8.0.1		14:5
22	http://127.8.0.1	GET	/vulnerabilities/xss_r/			200	4687	HTML		Vulnerability: Reflecte...			127.8.0.1		15:0
24	http://127.8.0.1	GET	/vulnerabilities/xss_r/			200	4687	HTML		Vulnerability: Reflecte...			127.8.0.1		15:0
25	http://127.8.0.1	GET	/vulnerabilities/xss_r/?name=%3C...		✓	200	4727	HTML		Vulnerability: Reflecte...			127.8.0.1		15:0
26	http://127.8.0.1	GET	/vulnerabilities/xss_r/?name=%3C...		✓	200	4737	HTML		Vulnerability: Reflecte...			127.8.0.1		15:1
17	http://127.8.0.1	GET	/vulnerabilities/xss_s/			200	5320	HTML		Vulnerability: Stored C...			127.8.0.1		15:0
21	http://127.8.0.1	POST	/vulnerabilities/xss_s/		✓	200	5416	HTML		Vulnerability: Stored C...			127.8.0.1		15:0
23	http://127.8.0.1	GET	/vulnerabilities/xss_s/			200	5416	HTML		Vulnerability: Stored C...			127.8.0.1		15:0

Request Details:

```

1 GET /vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27reflect%27%2A%3C%2Fscript%3E HTTP/1.1
2 Host: 127.8.0.1
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-GB,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
  
```

Response Details:

```

1 HTTP/1.1 200 OK
2 Date: Sun, 13 Sep 2026 09:40:38 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 X-XSS-Protection: 0
8 Vary: Accept-Encoding
9 Content-Length: 4388
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=utf-8
13
14
15 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
16 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
17 <html xmlns="http://www.w3.org/1999/xhtml">
18
  
```

Burp Suite Proxy — request/response for the Reflected XSS GET request, with the payload visible in the URL and reflected in the HTML response

The screenshot shows the Burp Suite interface with the Proxy tab selected. The HTTP history table lists several requests, with the selected request being a POST to `/vulnerabilities/xss_s/`. The detailed view of this request and response is shown below the table.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	
17	http://127.8.0.1	GET	/vulnerabilities/xss_s/			200	5320	HTML		Vulnerability: Stored C...			127.8.0.1		15:00:0813...	221
21	http://127.8.0.1	POST	/vulnerabilities/xss_s/		✓	200	5416	HTML		Vulnerability: Stored C...			127.8.0.1		15:03:1413...	119

Request Details:

```

1 POST /vulnerabilities/xss_s/ HTTP/1.1
2 Host: 127.8.0.1
3 Content-Length: 94
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-GB,en;q=0.9
9 Origin: http://127.8.0.1
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
  
```

Response Details:

```

1 HTTP/1.1 200 OK
2 Date: Sun, 13 Sep 2026 09:33:14 GMT
3 Server: Apache/2.4.25 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 5088
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13
14 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
15 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
16 <html xmlns="http://www.w3.org/1999/xhtml">
17 <head>
  
```

Burp Suite Proxy — request/response for the Stored XSS POST submission to the guestbook

3.3 Broken Authentication Testing

The DVWA login page and the Brute Force module were used to evaluate authentication controls. Testing included checking for account lockout after repeated failed login attempts, reviewing whether session

tokens were predictable, and confirming whether session cookies were transmitted with HttpOnly/Secure flags.

Findings: DVWA's default configuration does not enforce account lockout or rate-limiting, making it susceptible to brute-force and credential-stuffing attacks in a production-equivalent setting. This was further corroborated by the OWASP ZAP passive scan, which flagged the application's session cookie (PHPSESSID) handling under 'Session Management Response Identified' (see Section 4).

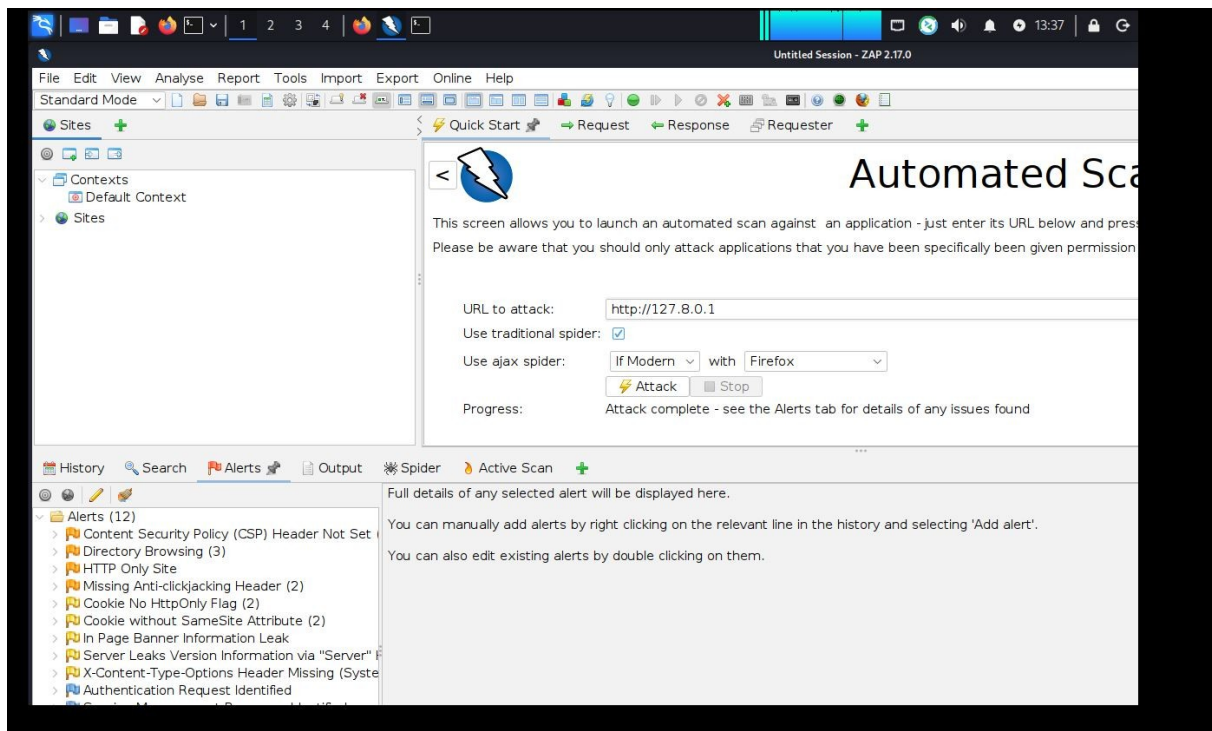
3.4 Security Misconfiguration Review

The application and server configuration were reviewed for common misconfiguration issues: verbose PHP error messages, directory listing, default credentials, outdated software versions, and missing HTTP security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options).

Findings: DVWA ships with default credentials (admin/password), permissive security levels, and PHP error reporting enabled — all of which are analogous to real-world misconfiguration issues found during assessments.

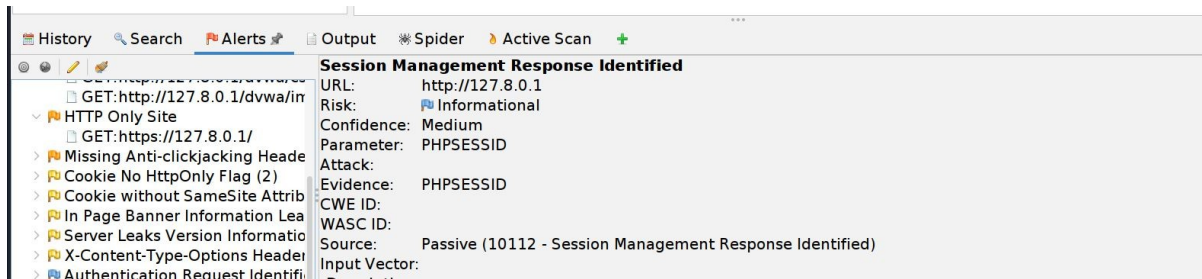
4. Automated Scanning with OWASP ZAP

An automated scan was run against the DVWA instance (<http://127.8.0.1>) using OWASP ZAP's Quick Start feature, which combines the traditional Spider (to crawl and enumerate all reachable pages/parameters) with an Active Scan (to test discovered parameters for known vulnerability patterns).

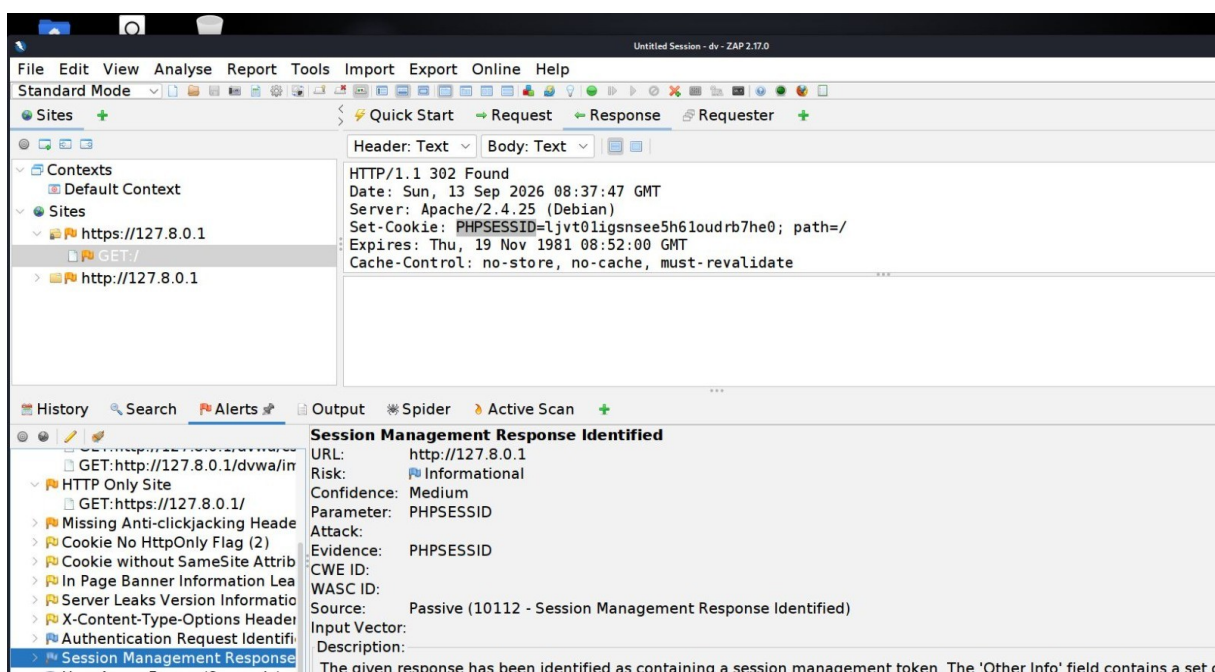


OWASP ZAP Quick Start — automated scan configured against <http://127.8.0.1>, attack complete, 12 alerts raised

The scan raised 12 alerts, including security misconfiguration issues (missing Content Security Policy header, directory browsing enabled, missing anti-clickjacking header, cookies without the HttpOnly/SameSite attributes) and informational findings related to session management and server version disclosure.



ZAP Alerts tab — 'Session Management Response Identified' finding, flagging the PHPSESSID cookie



ZAP Response view showing the Set-Cookie header for PHPSESSID alongside the corresponding alert detail

This finding is directly relevant to the Broken Authentication testing in Section 3.3: ZAP's passive scan independently confirmed that the application's session token handling lacks additional hardening, cross-validating the manual observation that DVWA does not enforce stronger session protections by default.

5. Findings & Risk Levels

The table below summarizes all confirmed findings, mapped to their OWASP Top 10 category and assigned a severity rating based on exploitability and potential impact.

#	Vulnerability	Location / Module	Severity	OWASP Category	Status
1	SQL Injection (Error-based)	DVWA - SQL Injection	Critical	A03: Injection	Confirmed

#	Vulnerability	Location / Module	Severity	OWASP Category	Status
2	Session Management Weakness (PHPSESSID)	DVWA - All pages (ZAP passive scan)	Medium	A07: Auth Failures	Confirmed
3	Reflected XSS	DVWA - XSS (Reflected)	High	A03: Injection	Confirmed
4	Stored XSS	DVWA - XSS (Stored)	Critical	A03: Injection	Confirmed
5	Broken Authentication	DVWA - Login / Brute Force	High	A07: Auth Failures	Confirmed
6	Security Misconfiguration	Server / DVWA config	Medium	A05: Misconfiguration	Confirmed
7	Missing Security Headers	HTTP responses (ZAP scan)	Low	A05: Misconfiguration	Confirmed

5.1 Severity Rating Criteria

- Critical — Directly exploitable, leads to full data compromise or system takeover (e.g., dumping the full user/password table).
- High — Exploitable with moderate effort, significant impact on confidentiality/integrity (e.g., session hijacking via XSS).
- Medium — Requires specific conditions to exploit, or has limited direct impact (e.g., misconfigurations that aid other attacks).
- Low — Informational or defense-in-depth issues (e.g., missing security headers).

6. Mitigation Methods

6.1 SQL Injection

- Use parameterized queries / prepared statements instead of string-concatenated SQL.
- Apply the principle of least privilege to database accounts used by the application.
- Validate and whitelist input formats; use an ORM where practical.
- Deploy a Web Application Firewall (WAF) as a defense-in-depth measure.

6.2 Cross-Site Scripting (XSS)

- Encode/escape all user-supplied output before rendering it in HTML, JS, or attribute contexts.
- Implement a strict Content Security Policy (CSP) to restrict script execution sources.
- Set the HttpOnly flag on session cookies so they cannot be read via JavaScript.
- Sanitize input server-side even when client-side validation is present.

6.3 Broken Authentication

- Enforce strong password policies and multi-factor authentication (MFA).
- Implement account lockout / rate-limiting after repeated failed login attempts.
- Use secure, random, and regularly rotated session tokens; invalidate sessions on logout.

- Avoid exposing session identifiers in URLs.

6.4 Security Misconfiguration

- Disable verbose error messages and stack traces in production.
- Remove default credentials and unused features/services.
- Keep all software, frameworks, and dependencies patched and up to date.
- Configure standard security headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS).

7. Conclusion

This VAPT exercise confirmed that DVWA — by design — contains critical, exploitable vulnerabilities across SQL Injection, XSS, Broken Authentication, and Security Misconfiguration categories. Manual testing with Burp Suite validated the precise request/response behavior behind each vulnerability, while OWASP ZAP's automated scan provided broader coverage and corroborated the manual findings. The exercise demonstrates why a layered VAPT approach — combining automated breadth with manual depth — is essential for accurately assessing the real-world security posture of a web application, and why the mitigation methods outlined above should be treated as baseline requirements rather than optional hardening steps.

8. References

- OWASP Top 10 — <https://owasp.org/www-project-top-ten/>
- OWASP ZAP Documentation — <https://www.zaproxy.org/docs/>
- PortSwigger Burp Suite Documentation — <https://portswigger.net/burp/documentation>
- DVWA GitHub Repository — <https://github.com/digininja/DVWA>